Hackage: https://hackage.haskell.org/package/exceptions-0.10.5

```
-- Control.Monad.Catch
class Monad m => MonadThrow m where
 throwM :: Exception e => e -> m a

class MonadThrow m => MonadCatch m where
 catch :: Exception e => m a -> (e -> m a) -> m a

class MonadCatch m => MonadMask m where
 mask :: ((forall a. m a -> m a) -> m b) -> m b
 uninterruptibleMask :: ((forall a. m a -> m a) -> m b)
-> m b
 generalBracket :: m a -> (a -> ExitCase b -> m a) -> (a
-> m b) -> m (b, c)

data ExitCase a =
 ExitSuccess a
 | ExitException SomeException
 | ExitAbort

data Handler m a = forall e. Exception e => Handler (e
-> m a)

catchAll :: MonadCatch m => m a -> (SomeException -> m
a) -> m a
catchIOError :: MonadCatch m => m a -> (IOError -> m a)
-> m a
catches :: MonadCatch m => m a -> [Handler a] -> m a
try :: (MonadCatch m, Exception e) => m a -> m (Either e
a)
onException :: MonadCatch m => m a -> m b -> m a
onError :: MonadMask m => m a -> m b -> m a
bracket :: MonadMask m => m a -> (a -> m c) -> (a -> m
b) -> m b
bracket_ :: MonadMask m => m a -> m c -> m b -> m b
finally :: MonadMask m => m a ->m b -> m b
bracketOnError :: MonadMask m => m a -> (a -> m c) -> (a
-> m b) -> m b
...
```

Instances

```
instance MonadThrow []
instance MonadThrow Maybe
instance MonadThrow IO
instance e ~ SomeException => MonadThrow (Either e)
...

instance MonadCatch IO
instance e ~ SomeException => MonadCatch (Either e)
```

```
instance MonadMask IO
instance e ~ SomeException => MonadMask (Either e)
```